

A Half Century of Formal Methods

Dines Bjørner¹ and Klaus Havelund^{2*}

¹ Technical University, Kgs. Lyngby, Denmark

² Jet Propulsion Laboratory, California Inst. of Tech., Pasadena, USA

Abstract. In this **45 Years of Formal Methods** review we first delineate what we mean by method, formal method, computer science, computing science, software engineering, and model-oriented and algebraic methods. Based on this we shall characterise a spectrum from specification-oriented methods to analysis-oriented methods. Then we shall provide an **Overview**: which are the ‘prerequisite works’ that have enabled formal methods; and which are, to us, the, by now, classical formal **Specification and Analysis Methods**. We then ask ourselves the question: **A Success Story?** Have formal methods for software development, in the sense of this paper been successful? Our answer is a guarded yes. At the start of **More Personal Observations**, we relate, in **The DDC Ada Story**, a 1980–1984 formal methods success story. We motivate the guarded answer by discussing **Some Obstacles to Formal Methods** in university research and education as well as in industry practice. Finally we discuss current programming language developments in a **Next 10 Years** perspective. Although their recent emergence has occurred in parallel with formal methods development, and seemingly independent thereof, we do see some convergence.

Introduction

The subject of this review is that of methods for developing trustworthy software. That is: software for which we can provide reasonably convincing arguments that they meet customer expectations and are correct wrt. some form of specification.

We see software development as (1) beginning, ideally, with a domain description, i.e., a specification of the application domain void of any reference to requirements let alone software, (2) proceeding to a requirements prescription, (3) moving to the specification of a software design, and (4) finally code. Typically, however, the first phase, domain engineering, is omitted. Each of these phases may involve both synthesis and analysis: creating and analysing a specification. We review such specification and analysis methods which are based on mathematics.

* The research by this author was carried out at the Jet Propulsion Laboratory, California Institute of Technology, under a contract with the National Aeronautics and Space Administration (80NM0018D0004).

Some Delineations

By a **method** we shall understand a set of **principles** for **selecting** and **applying techniques** and **tools** for **synthesizing** and/or **analyzing** an **artefact**. In this paper we shall be concerned with *methods for synthesizing and analysing software artefacts*. We consider the software code to be a *mathematical* artefact. That is why we shall only consider such methods which we call formal methods. By a **formal method** we shall understand a method whose **techniques** and **tools** can be explained in **mathematics**. If, for example, the method includes a **specification language**, then that language has a **formal syntax**, a **formal semantics**, and a **formal proof system**. The **techniques** of a formal method help **construct** a specification, and/or **analyse** a specification, and/or **transform (refine)** one (or more) specification(s) into a program. The **techniques** of a formal method, (besides the specification language) are typically supported by software. A central concern in the development of software, is to **reason** about properties of what is being developed. Among such properties are correctness of program code with respect to requirements and computing resource usage. Either some software is developed **systematically** [some arguments are made, but they are not necessarily formal, although they are in a form such that they can be made formal], or it is developed **rigorously** [some arguments are made and they are formal], or it is developed **formally** [all arguments are formal]. Boundary lines are, however, fuzzy. By **computer science** we shall understand the study of and knowledge about the mathematical structures that “exist inside” computers. By **computing science** we shall understand the study of and knowledge about how to construct those structures. The term **programming methodology** is here used synonymous with computing science. Software engineering is the actual pursuit of software development based primarily on computing science insight. By **engineering** we shall understand the design of technology based on scientific insight and the analysis of technology in order to assess its properties (including scientific content) and practical applications. Software engineering, to us, ideally entails the **engineering of domain descriptions** [9] ($\mathcal{D}$), the **engineering of requirements prescriptions** ($\mathcal{R}$), the **engineering of software designs** [and code] ($\mathcal{S}$), and the engineering of informal and formal relations between domain descriptions and requirements prescriptions: $\mathcal{R}$ is a model of $\mathcal{D}$, and domain descriptions, requirements prescriptions and software designs: $\mathcal{S}$ can be proved correct with respect to $\mathcal{R}$ in the context of $\mathcal{D}$. Our delineation of software engineering is based (i) on treating all specifications as mathematical structures, and, in addition to these programming methodological concerns, (ii) by also considering more classical engineering concerns. In preparation for two examples of formal specifications we narrate, i.e., informally specify, in **Example 0**, the data structure concept of a stack.

Example 0: Stacks

Entities:

1. There are stack ($s : S$) and element ($e : E$) entities;
2. there is an *is_empty* predicate; and
3. there are *empty*, *push*, *pop* and *top* operations.

Properties:

4. creating an *empty* stack creates an *is_empty* stack;
5. popping a stack which can be understood as a stack, s , on which is pushed an element, e , yields that stack s ;
6. inquiring as to the top element of a stack which can be understood as a stack, s , on which is pushed an element, e , yields that element e .

By an **algebraic specification**, see **Example 1**, we shall understand one whose data types, called carrier sets, are abstract, i.e., their elements are considered nullary operations, and whose postulated operations are defined in terms of their signatures and axioms over the operations.

Example 1: An Algebraic Specification of Stacks

types
 E, S
operation signatures
 $\text{is_empty}: S \rightarrow \mathbf{Bool}$
 $\text{empty}: S, \text{push}: E \times S \rightarrow S, \text{pop}: S \xrightarrow{\sim} S, \text{top}: S \xrightarrow{\sim} E$
axioms $\forall e : E, s : S \bullet$
 $\text{is_empty}(\text{empty}) = \text{true}$
 $\text{pop}(\text{push}(e, s)) = s$
 $\text{top}(\text{push}(e, s)) = e$

By a **model-oriented specification**, see **Example 2**, we shall understand one whose data types are concrete, such as numbers, sets, Cartesians, lists and maps, and whose operations are defined in terms of functions over these concrete data types.

Example 2: A Model-oriented Specification of Stacks

types
 $E, S = E^*$
operation signatures
 $\text{is_empty}: S \rightarrow \mathbf{Bool}$
 $\text{empty}: S, \text{push}: E \times S \rightarrow S, \text{pop}: S \xrightarrow{\sim} S, \text{top}: S \xrightarrow{\sim} E$
definitions $\forall e : E, s : S \bullet$
 $\text{is_empty}(s) \equiv s = \langle \rangle$
 $\text{empty} \equiv \langle \rangle$
 $\text{push}(e, s) \equiv \langle e \rangle^s$
 $\text{pop}(s) \equiv \mathbf{tail\ s\ pre} \neg \text{is_empty}(s)$
 $\text{top}(s) \equiv \mathbf{head\ s\ pre} \neg \text{is_empty}(s)$

Overview

What are the origins of formal methods? Here we should like to briefly mention some of the giant contributions which form a foundation. See Table 1.

Table 1: Giant Contributions

- **John McCarthy**[83, 84]: *Recursive Functions of Symbolic Expressions and Their Computation by Machines and Towards a Mathematical Science of Computation.*
- **Peter John Landin** and **Rodney Martineau [Rod] Burstall** [80, 20]: *The Mechanical Evaluation of Expressions, and Programs and their Proofs: an Algebraic Approach.*
- **Robert W Floyd** [43]: *Assigning Meanings to Programs.*
- **John Charles Reynolds**[103]: *Definitional Interpreters for Higher-order Programming Languages.*
- **Dana Stewart Scott** and **Christopher S. Strachey**[107]: *Towards a Mathematical Semantics for Computer Languages.*
- **Edsger Wybe Dijkstra** [36]: *A Discipline of Programming.*
- **Charles Anthony Richard Hoare**[62, 63]: *An Axiomatic Basis for Computer Programming and Proof of Correctness of Data Representations.*

Finally there are the concepts of **abstract interpretation** and **partial evaluation**. Beginning with the work of [33, 30, 28, 31, 32, 29, Cousot & Cousot] abstract interpretation is at the foundation of not only static program analysers but well-nigh any form of program interpretation. The work of Neil Jones et al., [74], beautifully illustrates the power of considering programs as formal, mathematical objects.

Some monographs or text books “in line” with formal development of programs, but not “keyed” to specific notations, are listed in Table 2.

Table 2: Seminal Text Books

- **Knuth**: *The Art of Programming* [77].
- **Dijkstra**: *A Discipline of Programming* [36].
- **Gries**: *The Science of Programming* [47].
- **Reynolds**: *The Craft of Programming* [104].
- **Hehner**: *The Logic of Programming* [60].

Specification and Analysis Methods

Early formal methods focused primarily on specification writing, and less on machine assisted analysis. Later formal methods, such as theorem provers and

model checkers, gave analysis high priority. A representative list is shown in Table 3.

Table 3: Some Formal Methods and Tools

- 1974 **VDM** [*Vienna Development Method*] model-oriented [12, 13, 73, 41, 42] [en.wikipedia.org/wiki/Vienna_Development_Method#External_links].
- 1977 **Z** [*Z: Zermelo*] model-oriented [114, 109, 58] [zuser.org].
- 1980 **SPIN/Promela** model checker [65] [spinroot.com/spin].
- 1980s **B** [*B: Bourbaki*] model-oriented with emphasis on proof [1] [methode-b.com].
- 1989 **Coq** [*Galina*] type-theoretic theorem prover [66, 6] [coq.inria.fr].
- 1986/1990 **Isabelle/HOL** [*Higher Order Logic*] classical logic theorem prover [94] [isabelle.in.tum.de].
- 1992 **RAISE** [*Rigorous Approach to Ind. SE*] model- and property-oriented [46, 8].
- 1990 **ASM** [*Abstract State Machines*] model-oriented, classical mathematics [48–50, 101].
- 1990 **PVS** [*Prototype Verification System*] classical logic theorem prover [97, 98] [pvs.csl.sri.com].
- Mid 1990s **ACL2** [*A Comp. Logic for [Appl.] Common Lisp*] classical logic theorem prover [76].
- 1994 **STeP** [*Stanford Temporal Prover*] temporal logic for reactive systems, theorem prover [82, 15].
- 1995 **Uppaal** [*Uppsala/Aalborg*] timed automata, model-checker [45, 5].
- 1997 **Alloy** model-oriented [69, 68], incorporates a *satisfiability* tool.
- 2000 **[nu]SMV** [*Symbolic Model Verifier*] symbolic model-checker [22].
- 2004 **Astrée** anstract interpretation [17], a code analyzer.
- 2008 **Z3** [*Zermelo*] [91, 16], is a *satisfiability modulo theories* tool.
- 2009 **Event B** [2], incorporates a *theorem prover* and is embedded in the **Rodin** development platform.

Yuri Gurevitch’s **Abstract State Machines** [48, 49, 101] (**ASM**), also known as ‘Evolving Algebras’, was developed as a theory for computation, and did not have a fully formal syntax and was not tool supported. This work can be seen as advocating the use of mathematics for specifying systems. **AsmL (Abstract State Machine Language)** [50] was later developed with full formal syntax and tool support [archive.codeplex.com/?p=asm1], and used successfully at Microsoft for model-based testing. In a similar vein, Wolfgang J. Paul [100, 92, 99, 26] has advocated using mathematics as a specification language.

Shallow program analysis is provided by a number of *static analysis* tools – such as **Semmlé** [semmlé.com], **Coverity** [coverity.com], **CodeSonar** [gram-matech.com/codesonar] and **KlocWork** [klocwork.com]. These static analysers scale to large programs, are automated, and are, from an industrial point, very useful. However, this is at the prize of the limited properties they can check;

they can usually not check functional properties: that a program satisfies its requirements.

Satisfiability Modulo Theories [SMT] solving has become a major development in formal methods, used in many development tools. **Z3** is such an SMT analysis tool, and appears to be a leading tool with 24,000 downloads in 2017 and 5,000+ citations of [16]. **SMT LIB** provides a common input language for SMT solvers and standard rigorous descriptions of background theories used in SMT systems [smtlib.cs.uiowa.edu]. The **SMT LIB** initiative is remarkable in having united very many researchers and developers of SMT systems around one notation.

The foremost property-oriented (algebraic) formal methods (alphabetically listed) are listed in Table 4.

Table 4: Some Algebraic Specification Languages

- 1996 **Maude** [88, 87, 23] [maude.cs.illinois.edu/w/index.php/The_Maude_System].
- 1997 **CafeOBJ** [44] [cafeobj.org].
- 1998 **CASL** [Common Algebraic Spec. Language] [25] [informatik.uni-bremen.de/cofi].

The definitive text on algebraic semantics is

- Sanella & Tarlecki’s **Foundations of Algebraic Semantics and Formal Softw. Devt.** [105].

It is a characteristic of algebraic methods that their specification logics are analysis friendly, usually in terms of rewriting.

A special category of systems is that of **reactive** systems. A reactive system consists of a collection of individual components executing independently, in parallel, while exchanging messages. Some formal notations for describing reactive systems are listed in Table 5.

Table 5: Reactive Systems Languages

- **CSP** [Communicating Sequential Processes] [64] [wotug.org].
- **CCS** [Calculus of Communicating Systems] for modelling concurrence [89] [en.wikipedia.org/wiki/Calculus_of_communicating_systems].
- **DC** [Duration Calculus] [117] for modelling time-continuous properties [en.wikipedia.org/wiki/Duration_calculus].
- **MSC** [Message Sequence Charts] [67] for graphically modelling message communication between simple processes [itu.int/rec/T-REC-Z.120].
- **Petri Nets** [102] for modelling arbitrary synchronisation of multiple processes [http://www.informatik.uni-hamburg.de/TGI/PetriNets/-index.php].

- **Statecharts** [51] for modelling hierarchical systems [statecharts.github.io].
- **TLA+** [Temporal Logic of Actions] [79] for modelling temporal properties [lamport.azurewebsites.net/tla/tla.html].

An abundance of regular workshops, symposia and conferences have grown up around formal methods. See Table 6.

Table 6: Regular Events

- **VDM** and **FME** [FM Europe] symposia [10].
- **Z**, **B**, **ZB**, **ABZ**, etc. meetings, workshops, symposia, conferences, etc. [18].
- **SEFM** [Softw.Eng. and Formal Methods] [78].
- **ICFEM** [Intl.Conf. of Formal Engineering Methods] [37].
- **IFM** [Integrated Formal Methods] [75].

Although some of these conferences started out as specification-oriented, today they are all more or less analysis-oriented. The main focus of research today is analysis. See Table 7 for highly analysis-oriented events:

Table 7: Regular Analysis-focused Events

- **CAV** [Computer Aided Verification].
- **TACAS** [Tools and Algorithms for the Construction and Analysis of Systems].
- **CADE** [Conference on Automated Deduction].

Table 8 lists major formal methods journals.

Table 8: Formal Methods Journals

- **Formal Aspects of Computing**
[link.springer.com/journal/165],
- **Formal Methods in System Design**
[link.springer.com/journal/10703] and
- **Software Tools for Technology Transfer**
[www.springer.com/computer/swe/journal/10009];

Integrated Formal Methods

Formal methods are sometimes complemented by some of “the related” formal notations. The **RAISE** Specification Language, **RSL**, includes **CSP** and some restricted notion of **object-orientedness** and a subset of **RSL** has been extended with **DC**[56, 57]. **VDM** and **Z** has each been extended with some (wider) notion of **object-orientedness**: **VDM++** [38], respectively **object Z** [113, 34]. A general shortcoming of all the above-mentioned formal methods is their inability to

express continuity in the sense, at the least, of first-order differential calculus. The **IFM** conferences [75] focus on such “integrations”. Haxthausen [56] outlines integration issues for model-oriented specification languages. **Hybrid CSP** [59, 116] is **CSP** + differential equations. A recent development is that of **rTiMo** [115], a variant of **CSP** with **real Time** and **Mobility**!

A Success Story ?

With all these books, publications and conferences can we claim that formal methods have become a success — an integral part of computing science and software engineering ? and established in the software industry ? Our answer is now a qualified yes ! In 2014 [11] it was no ! Although formal methods have yet to become an integral part of computing science, software engineering and software industry, it is easier today to say that progress has been made. The next sections put forward some of the qualifications of the yes.

Personal Observations

There are different possible ways to analyze the situation of formal methods and their adoption by industry. These include (a) comparing the various methods, holding them up against one another, (b) evaluating which application areas they are suited for, (c) identifying gaps in them, and (d) focusing on “soft” topics such as educational, psychological, and process oriented issues. Rather than going into technicalities, we shall in the following discuss a higher level of abstraction “hindrances to formal methods” which seems common to all formal methods, and focus mostly on (d) and to some limited extent (c). But first a small “detour” !

The DDC Ada Success Story

In 1980 a team of six just-graduated MScs started the industrial development of a commercial **Ada** compiler. Their (MSc theses) semantics description (in **VDM+CSP**) of **Ada** were published Springer [14]. The project took some 44 man years in the period 1 Jan. 1980 to 1 Oct. 1984 – when the US DoD, in Sept. 1984, had certified the compiler. The six initial developers were augmented by 3 also just-graduated MScs in 1981 and 1982. The “formal methods” aspects of the development approach was first documented at ICS’77 [7]. The project staff were all properly educated in formal semantics and compiler development in the style of [7], [12] and [13]. The completed project was evaluated in [24] and in [96]. Now, 35 years later, mutations of that 1984 **Ada** compiler are still around ! From having taken place in Denmark, a core **DDC Ada** compiler product group was moved to the US in 1990 [Cf. **DDC-I Inc.**, Phoenix, Arizona <http://www.ddci.com/>] — purely based on marketing considerations. Several versions of **Ada** has been assimilated into the 1981–1984 design. Several generations of less ‘formal methods’-trained developers have worked and are working on the **DDC-I Inc. Legacy Ada** compiler systems. For the first 10 years of the 1984 **Ada** compiler product less than one man month was spent per year on

corrective maintenance – dramatically below industry “averages”! The DDC Ada development was systematic: it had roughly up to eight (8) steps of “refinement”: two (2) steps of domain description of Ada (approx. 11.000 lines), via four (4) steps of requirements prescription for the Ada compiler (approx. 55.000 lines), and two (2) steps of design (approx. 6.000 lines) and coding of the compiler itself. Figure 1 reflects the Ada compiler development graph.

- A The theory of the contents of the triplet of top left boxes is covered by McCarthy, Scott and Strachey[83, 84, 107].
- B The use of CSP is due to Hoare [64].
- C The ‘First Order Semantics’ of is dealt with in Landin and Reynolds[80, 103].
- D The ‘Imperative Stack and Macro-expansion Semantics’ ideas, originated with Bekič[72].
- E The ‘A Code’ to ‘Compiling Algorithm’ idea, was that of McCarthy & Painter[85].

The whole transgression A–E was reported in 1977 [7]. The newly graduated students were well-versed in these papers. Throughout the emphasis was on formal specification. No attempt was really made to express, let alone prove, formal properties of any of these steps nor their relationships. The formal/systematic use of VDM resulted in less than 1% of the original development costs being spent between 1985 and today on error-correction, and must hence be said to be an unqualified formal methods success story.

• • •

The following personal observations can be seen in the context of the more than 35 years old DDC Ada compiler project.

Formal Method Challenges

We remind the reader of our characterization of what constitutes a formal method scenario. At its core, we have two artifacts, an abstract specification, $\mathcal{A}$, and a more concrete implementation, $\mathcal{C}$, the latter purported to be an implementation of the former. That entails the verification, e.g. by formal testing, model checking, or formal proof, that $\mathcal{C}$ correctly implements $\mathcal{A}$. An example is a program about which we wish to verify some assertions.

• **Creating Multiple Specifications Take Time** Developers may consider it too time consuming to produce a formal specification $\mathcal{A}$ (say a formal requirements prescription) in addition to the implementation $\mathcal{C}$. This is in particular the case if specification $\mathcal{A}$ is written in a different language than the implementation $\mathcal{C}$. There have been success stories with this approach. The first author has e.g. had good experiences with this approach during the afore mentioned development of the Ada compiler, which was formalized in VDM before being implemented. There is also the use of **TLA+** at Amazon Web Services [93], which is considered a successful attempt in this direction. One may also observe that

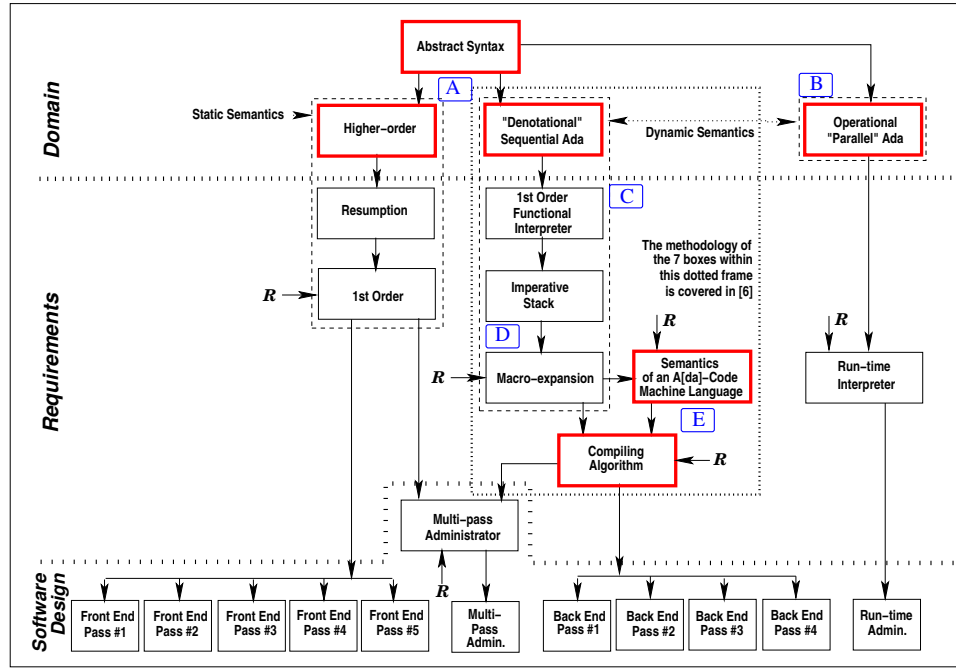


Fig. 1. The DDC Ada compiler development graph

theoretical computer scientists do use mathematics to develop theories before these theories are implemented in code. These formalisations are, however, usually written in free style mathematical notation (targeted for publications), and not in formal languages. There have also been successful applications of formal methods to analyze existing systems [52, 54], exposing otherwise very hard to find bugs. In general, though, it is probably fair to say, that these formalisations beyond the code are few and far in between, and that industrial developers seldom write formal stand-alone specifications (except for assertions in the code). Even formal methods people usually do not write specifications in formal machine checked languages (beyond the free style theories) before they develop code. The main “problem” is that the distance between specification and implementation is relatively small compared to other engineering disciplines, and, as we shall illustrate below, it is getting smaller and smaller as programming languages evolve. We are not judging “right or wrong”, but rather observing behavior patterns.

- **Proofs Take Time** It is a fact, that proving that an implementation implements a specification usually cannot be fully automated. Although model checking is automated, it does not scale to large systems. In deductive theorem proving the main problem is loop invariants, invariants which a user has to in-

vent, like lemmas in a proof. SMT solving has been a big advance in formal methods, but still does not solve the loop invariant discovery problem.

- **A “Lack of Formal Methods Research and Education” Obstacle** There is not enough research into and teaching of formal methods. Just because a formal method may be judged to not yet be industry-scale is no hindrance to it being researched and — we must prepare our students properly. This obstacle is of “history-of-science-and-engineering” nature.

- **A “No Standards” Obstacle** It seems to be a fact that industry will not use a formal method unless it is standardised. Most formal method specification languages were conceived and developed by small groups of usually university researchers. This basically stands in the way of preparing for standards and for developing and later maintaining tools.

- **An “Intra-Departmental” Obstacle** There are two facets to this obstacle.

- ◊ We find that core courses in computing science and software engineering are often not explained to students in terms of mathematics and formal methods.

- ◊ And we find that scientific papers on methodology are either not written, or, when written and submitted are passed over by referees who do not understand the difference between computer science and computing science.

It is claimed that the teaching and research staff of departments of computer science and software engineering are generally unaware of the science & engineering aspects of each others’ individual sub-fields. That is, we often see software engineering researchers and teachers unaware of the disciplines of, for example, *Mathematical Logic*, *Recursive Function Theory*, and *Abstraction and Modelling* (eg., *Formal Methods*) — with the unawareness manifesting itself in the lack of use of cross-discipline techniques and tools. Such a lack of unawareness of intra-department disciplines seems rare among mathematicians. Physicists freely avail themselves of most forms of classical mathematics; and so do engineering researchers and engineers.

- **A “Slide in Professionalism” Obstacle:** There is an “Education Gap” in the software industry: between many “newly” graduated programmers and the “older” staff programmers. Some graduates have learned and, to some extent, master, various aspects of formal methods, while older staff are unaware of formal methods, and continue using “old” practices. The end result is that new staff “fall back” on the “older” ways of software development, i.e., are basically barred from using formal methods.

The Next 10 Years

No-one can predict the future. However, we shall point out some trends that we are observing, trends that on the one hand are easy to observe, but, on the other hand, deserve to be highlighted.

Among the earlier mentioned obstacles, we mentioned developers’ resistance to formulate artifacts numerous times and in different languages, and we mentioned the challenging verification effort. If everything is performed within the same language framework, perhaps these obstacles could be torn down.

We see two somewhat independent trends, which together potentially can change the way formal methods are used. The first trend is the emergence of new programming languages with high-level features that are similar to those found in specification languages. The second trend is the emergence of verification tools for programming languages. One can say that the programming language community and the formal methods community are converging towards a *point of singularity*, where specification and programming can be done within the same language. These two trends together may break down the barrier for programmers to write specifications. We start by summarizing the desirable characteristics of early specification languages.

Early Specification Languages: VDM, its object-oriented extension VDM++, and RSL are so-called wide-spectrum specification languages, including specification constructs as well as programming constructs. Specification constructs include design-by contract concepts such as pre- and post-conditions and (class) invariants (predicates that must hold on a collection of variables). Furthermore, types can be defined as set comprehensions (predicate sub-types), e.g. natural numbers can be defined as a sub-type of integers. A key feature is general predicate logic, allowing e.g. universal and existential quantification over infinite as well as finite sets, and the “creation” of infinite sets.

Other more programming oriented abstractions include the e.g. the merge of object-oriented and functional programming, algebraic data-types, also referred to as “rich enumeration types” where alternatives in an enumerated type can carry data, and pattern matching over such in e.g. switch statements, collections, such as sets, lists and maps, and iterators over such, allowing e.g. for loops over elements in a collection, and comprehension expressions, corresponding to set comprehension in set theory. In addition to these linguistic concepts, deductive verification systems for these languages were developed, although very manual and low level of nature, if mechanized at all.

Trend 1: High-level Programming Languages: The first trend is the design of new programming languages that adapt many of the above mentioned abstractions found in early wide-spectrum specification languages. Programming languages move towards what could be called *wide-spectrum programming languages*, to turn the original term *wide-spectrum specification language* on its head. Early examples in this trend include the elegant (mostly) functional programming language **ML** [90], combining functional and imperative programming, its object-oriented derivative **OCaml** [ocaml.org], and the purely functional language **Haskell** [111] [haskell.org]. These languages have, however, to a large extent unfairly been considered academic of nature, and have not hit main stream.

However, the trend has also taken place in main stream application programming languages that rely on automated garbage collection. **Java** [108] was one of the first programming languages to support collections such as sets, lists, and maps, provided as part of the standard library, as well as iterators, e.g. permitting for-loops over such. **Python** [python.org/psf] is another example of a main stream language combining object-orientation and some form of functional programming, as well as built-in succinct notation for sets, lists and maps, and

iterators over these, as well as list and map comprehensions. These languages were, however, not fully supporting concepts such as algebraic data types and pattern matching. The next generation of programming languages offer these features. E.g. **Scala** [scala-lang.org] and **Swift** [swift.org] are both examples of newer garbage collected application programming languages, that generally provide all of the above mentioned programming oriented features present of early specification languages. The trend is also seen in systems programming, where **Rust** [rust-lang.org] is probably the latest such example.

Trend 2: Verifiable Programming Languages: The second trend is an increased focus on providing verification support for programming languages. Early theoretical efforts in program correctness, such as the works of Hoare [62, 63] and Dijkstra [36], did indeed focus on correctness of programs, but this work formed the underlying theories and did not immediately result in tools. The trend we are pointing out is a tooling trend. It is the step-wise realization of Hoare’s grand verification challenge [61] “*the construction and application of a verifying compiler that guarantees correctness of a program before running it*”. Examples of such program verifiers are numerous at this point. They include verifiers for existing programming languages, including software model checkers such as **Java PathFinder (JPF)** [55] [javapathfinder.sourceforge.net] and **Bandera** [27] [bandera.projects.cs.ksu.edu/] for Java; and **SLAM** [openslam-org.github.io], **CBMC** [cprover.org/cbmc], and **Modex** [65] [spinroot.com/modex] for C. A key trend has been deductive theorem proving systems for programming languages, including **KeY**: [3] [key-project.org], based on the **JML** design-by contract specification language [openjml.org] for **Java**; **Verifast** [70] [github.com/verifast], **VVC** [jvet.hhi.fraunhofer.de] and the general analysis framework **Frama-C** [frama-c.com] for C; **Spark** for **Ada** [86] [adacore.com], and **Spec#** for **C#** [microsoft.com/en-us/research/project/spec]. But languages are also being born with verification in mind, including **Eiffel** [eiffel.-com], **Dafny** [81], **Whiley** [whiley.org], **P** [35] [github.com/p-org/P], **Why3** [40] [why3.lri.fr], as well as languages supporting value dependent types (a form of predicate subtypes), such as **Agda** [95] [wiki.portal.chalmers.se/agda/pmwiki.php] and **Idris**: [19] [idris-lang.org]. The **ACL2** theorem prover should be mentioned as a very early example of a verification system associated with a programming language, namely **LISP**. Common for many of these languages is their support for some form of design-by contract language, including pre- and post-conditions for example.

Some Other Trends: Although verification frameworks may be part of programming IDEs, one must acknowledge, that proving programs correct still is a challenge, even for the skilled formal methods expert. Testing will therefore for a while remain to be the most practical approach to ensure the correctness of real-sized applications. The merge of specification and programming language fits well with the concept of agile programming where your first prototype may be your specification, which you may refine and later use as a test oracle. Particular interesting are topics such as model-based testing [112], where formal models are used to generate tests, and runtime verification [53, 4], where pro-

gram executions are monitored and verified against formal specifications, e.g. expressed in variants of temporal logic or state machines.

There are two other directions that we would like to mention: visual languages and Domain-Specific Languages (DSLs). Formal methods have an informal companion in the model-based programming community, represented for example most strongly by **UML** [71] and its derivations. This form of modeling is graphical by nature. **UML** is often criticized for lack of formality, and for posing a linkage problem between models and code. However, visual notations clearly have advantages in some contexts. The typical approach is to create visual artifacts (for example class diagrams and state charts), and then derive code from these. An alternative view would be to allow graphical rendering of programs using built-in support for user-defined visualization, both of static structure as well as of dynamic behavior. This would tighten connection between lexical structure and graphical structure.

We also need powerful and simple-to-use capabilities of extending programming languages with new DSLs. Such are often referred to as internal DSLs. This will be critical in many domains, where there are needs for defining new DSLs, but at the same time a desire to have the programming language be part of the DSL to maintain expressive power. **Racket** [39] [racket-lang.org] is an example of an extensible programming language. The point of singularity is the point where developments in programming languages and formal methods converge, and specification, programming and verification are performed in an integrated manner, within the same language framework, additionally supported by visualization and meta-programming.

In contrast to the internal DSL approach just outlined above, the Language-Driven Engineering approach [110] advocates the use of external, typically graphical, DSLs. An external DSL comes with its own syntax that is completely independent of any general purpose programming language. **Actulus** [21] is an example of an external DSL for actuarians. Whereas the typical user of an internal DSL will be a programmer, a user of an external DSL may be an application expert without programming knowledge. We see both approaches as important.

Conclusion

We have reviewed facets of formal methods, discussed obstacles to their propagation, and discussed some possible future developments. We do express optimism that formal methods will overcome these obstacles! Computer and computing science research include many subfields, as mentioned in this article, which interact in various ways. These include e.g. programming languages, specification languages, theorem proving, model checking, formal testing, static analysis, dynamic analysis, visualization, meta-programming, and domain-specific languages. We see this interaction of technologies to grow and lead to interesting solutions. Artificial intelligence is a topic that we have not touched upon, but which undoubtedly will have impact on the software development field.

References

1. Jean-Raymond Abrial. *The B Book: Assigning Programs to Meanings*. Tracts in Theoretical Computer Science. Cambridge University Press, Cambridge, England, 1996.
2. Jean-Raymond Abrial. *Modeling in Event-B: System and Software Engineering*. Cambridge University Press, Cambridge, England, 2009.
3. Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt, and Mattias Ulbrich, editors. *Deductive Software Verification - The KeY Book - From Theory to Practice*, volume 10001 of *Lecture Notes in Computer Science*. Springer, 2016.
4. Ezio Bartocci, Yliès Falcone, Adrian Francalanza, Martin Leucker, and Giles Reger. Introduction to runtime verification. In Ezio Bartocci and Yliès Falcone, editors, *Lectures on Runtime Verification - Introductory and Advanced Topics*, volume 10457 of *LNCS*, pages 1–23. Springer, 2018.
5. G. Behrmann, A. David, and K.G. Larsen. A Tutorial on Uppaal. In Corradini F. Bernardo M., editor, *Formal Methods for the Design of Real-Time Systems*, volume 3185 of *Lecture Notes in Computer Science*, pages 200–236, Berlin, Heidelberg, 2004. Springer.
6. Yves Bertot and Pierre Castéran. *Interactive Theorem Proving and Program Development. Coq'Art: The Calculus of Inductive Constructions*. EATCS Series: Texts in Theoretical Computer Science. Springer, 2004.
7. Dines Bjørner. Programming Languages: Formal Development of Interpreters and Compilers. In *International Computing Symposium 77 (eds. E. Morlet and D. Ribbens)*, pages 1–21. European ACM, North-Holland Publ.Co., Amsterdam, 1977.
8. Dines Bjørner. *Software Engineering, Vol. 1: Abstraction and Modelling; Vol. 2: Specification of Systems and Languages; Vol. 3: Domains, Requirements and Software Design*. Texts in Theoretical Computer Science, the EATCS Series. Springer, 2006.
9. Dines Bjørner. *Domain Science & Engineering – A Foundation for Software Development*. EATCS Monographs in Theoretical Computer Science. Springer, 2021.
10. Dines Bjørner et al., editors. *VDM, FME and FM Symposia 1987–2012*, Lecture Notes in Computer Science (LNCS) vols.:
 1. VDM'87: Belgium, March 1987; #252.
 2. VDM'88: Ireland, Sept. 1988; #328;
 3. VDM'90: Germany, April 1990, #428.
 4. VDM'91: Netherlands, Oct. 1991, #551-552.
 5. FME'93: Denmark, March 1993, #670.
 6. FME'94: Spain, October 1994; #873;
 7. FME'96: UK, March 1996, #1051;
 8. FME'97: Austria, Sept. 1997, #1313;
 9. FM'99: France, Sept. 1999; #1708–1709;
 10. FM'01: Germany, March 2001; #2021;
 11. FM'02: Denmark, July 2002; #2391;
 12. FM'03: Italy, Sept. 2003; #2805,
 13. FM'05: UK, July 2005; #3582;
 14. FM'06: Canada, August 2006; #4085;
 15. FM'08: Finland, May 2008; #5014;
 16. FM'09: Netherlands, Nov. 2009; #5850;
 17. FM'11: Ireland, June 2011; #6664;
 18. FM'12: France, Aug. 2012, #7436;
 . Springer, 1987–2012.

11. Dines Bjørner and Klaus Havelund. 40 Years of Formal Methods — 10 Obstacles and 3 Possibilities. In *FM 2014, Singapore, May 14-16, 2014*. Springer, 2014. Distinguished Lecture. imm.dtu.dk/~dibj/2014/fm14-paper.pdf.
12. Dines Bjørner and Cliff B. Jones, editors. *The Vienna Development Method: The Meta-Language*, volume 61 of *LNCS*. Springer, 1978. This was the first monograph on *Meta-IV*.
13. Dines Bjørner and Cliff B. Jones, editors. *Formal Specification and Software Development*. Prentice-Hall, 1982.
14. Dines Bjørner and Ole N. Oest, editors. *Towards a Formal Description of Ada*, volume 98 of *LNCS*. Springer, 1980.
15. Nikolaj Bjørner, Anca Browne, Michael Colon, Bernd Finkbeiner, Zohar Manna, Henny Sipma, and Tomas Uribe. Verifying Temporal Properties of Reactive Systems: A STeP Tutorial. *Formal Methods in System Design*, 16:227–270, 2000.
16. Nikolaj Bjørner, Ken McMillan, and Andrey Rybalchenko. Higher-order Program Verification as Satisfiability Modulo Theories with Algebraic Data-types. In *Higher-Order Program Analysis*, June 2013. <http://hopa.cs.rhul.ac.uk/files/proceedings.html>.
17. Bruno Blanchet, Patrick Cousot, Radhia Cousot, Laurent Mauborgne Jerome Feret, Antoine Miné, David Monniaux, and Xavier Rival. A static analyzer for large safety-critical software. In *Programming Language Design and Implementation*, pages 196–207, 2003.
18. Jonathan Bowen et al., editors. *Z, B, ZUM, ABZ Meetings, Conferences, Symposia and Workshops*, Z Users Workshops: 1986–1995; Z, ZB and ABZ Users Meetings: 1996–2013 Springer LNCS 1212, 1493, 1878, 2272, 2651, 3455, 5238, 5977 and 7316., 1986–2014.
19. Edwin C. Brady. Idris, a general-purpose dependently typed programming language: Design and implementation. *Journal of Functional Programming*, 23:552–593, 2013.
20. Rodney M Burstall and Peter J Landin. Programs and their proofs: an algebraic approach. Technical report, DTIC Document, 1968.
21. David R. Christiansen, Klaus Grue, Henning Niss, Peter Sestoft, and Kristján S. Sigtryggsson. Actulus Modeling Language – An actuarial programming language for life insurance and pensions. Technical Report, edlund.dk/sites/default/files/Downloads/paper_actulus-modeling-language.pdf, Edlund Inc., Denmark, 2015.
22. Edmund M. Clarke, Orna Grumberg, and Doron A. Peled. *Model Checking*. The MIT Press, Five Cambridge Center, Cambridge, MA 02142-1493, USA, January 2000. ISBN 0-262-03270-8.
23. Manuel Clavel, Francisco Durán, Steven Eker, Patrick Lincoln, Narciso Martí-Oliet, José Meseguer, and Carolyn Talcott. *All About Maude – A High-Performance Logical Framework*. Springer LNCS Vol. 4350, 2007.
24. Geert Bagge Clemmensen and Ole N. Oest. Formal specification and development of an Ada compiler – a VDM case study. In *Proc. 7th International Conf. on Software Engineering*, 26.-29. March 1984, Orlando, Florida, pages 430–440. IEEE, 1984.
25. CoFI (The Common Framework Initiative). *CASL Reference Manual*, volume 2960 of *Lecture Notes in Computer Science (IFIP Series)*. Springer-Verlag, 2004.
26. Ernie Cohen, Wolfgang J. Paul, and Sabine Schmaltz. Theory of multi core hypervisor verification. In Peter van Emde Boas, Frans C. A. Groen, Giuseppe F. Italiano, Jerzy Nawrocki, and Harald Sack, editors, *SOFSEM 2013: Theory and Practice of Computer Science*, volume 7741 of *Lecture Notes in Computer Science*, pages 1–27, Berlin, Heidelberg, 2013. Springer Berlin Heidelberg.

27. J. C. Corbett, M. B. Dwyer, J. Hatcliff, S. Laubach, C. S. Pasareanu, Robby, and Hongjun Zheng. Bandera: extracting finite-state models from java source code. In *Proceedings of the 2000 International Conference on Software Engineering. ICSE 2000 the New Millennium*, pages 439–448, June 2000.
28. P. Cousot. Semantic Foundation of Program Analysis. In S.S. Muchnick and N.D. Jones, editors, *Program Flow Analysis: Theory and Applications*, chapter 10, pages 303–342. Prentice-Hall, 1981.
29. P. Cousot. Abstract Interpretation. *ACM Computing Surveys*, 28(2):324–328, 1996.
30. P. Cousot and R. Cousot. Systematic Design of Program Analysis Frameworks. In *6th POPL: Principles of Programming and Languages*, pages 269–282. ACM Press, 1979.
31. P. Cousot and R. Cousot. Higher-order Abstract Interpretation (and application to comportment analysis generalising strictness, termination, projection and PER analysis of functional languages). In *1994 ICCL*, pages 95–112. IEEE Comp. Sci. Press, 1994.
32. P. Cousot and R. Cousot. Formal Language, Grammar and Set-constraint-based Program Analysis by Abstract Interpretation. In *7th FPCA*, pages 170–181. ACM Press, 1995. June.
33. Patrick Cousot and Rhadia Cousot. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *4th POPL: Principles of Programming and Languages*, pages 238–252. ACM Press, 1977.
34. John Derrick and Eerke A. Boiten. *Refinement in Z and Object-Z: Foundations and Advanced Applications*. Springer Publishing Company, Incorporated, 2nd edition, 2013.
35. Ankush Desai, Vivek Gupta, Ethan Jackson, Shaz Qadeer, Sriram Rajamani, and Damien Zufferey. P: Safe asynchronous event-driven programming. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '13)*, pages 321–332, New York, NY, USA, 2013. ACM.
36. Edsger Wybe Dijkstra. *A Discipline of Programming*. Prentice-Hall, 1976.
37. Jin Song Dong and et.al., editors. *International Conferences on Formal Engineering Methods*. IEEE Computer Society Press and Springer, 1997–2019.
38. Eugène H. Dürr and J. van Katwijk. VDM⁺⁺, A Formal Specification Language for Object Oriented Designs. In *COMP EURO 92*, pages 214–219. IEEE, May 1992.
39. Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi, Eli Barzilay, Jay McCarthy, and Sam Tobin-Hochstadt. A Programmable Programming Language. *Communications of the ACM*, 61(3):62–71, mar 2018.
40. Jean-Christophe Filliâtre and Andrei Paskevich. Why3 — where programs meet provers. In Matthias Felleisen and Philippa Gardner, editors, *Proceedings of the 22nd European Symposium on Programming*, volume 7792 of *Lecture Notes in Computer Science*, pages 125–128. Springer, March 2013.
41. John Fitzgerald and Peter Gorm Larsen. *Developing Software Using VDM-SL*. Cambridge University Press, Cambridge, UK, 1997.
42. John Fitzgerald and Peter Gorm Larsen. *Modelling Systems – Practical Tools and Techniques in Software Development*. Cambridge University Press, Cambridge, UK, Second edition, 2009.
43. Robert W. Floyd. Assigning Meanings to Programs. In *[106]*, pages 19–32, 1967.
44. Kokichi Futatsugi and Razvan Diaconescu. *CafeOBJ Report The Language, Proof Techniques, and Methodologies for Object-Oriented Algebraic Specification*. AMAST Series in Computing – Vol. 6. World Scientific Publishing Co. Pte. Ltd., 1998.

45. Behrmann G., Bengtsson J., David A., Larsen K.G., Pettersson P., and Yi W. Uppaal Implementation Secrets. In Damm W. and Olderog E.R., editors, *Formal Techniques in Real-Time and Fault-Tolerant Systems*, volume 2469 of *Lecture Notes in Computer Science*, pages 3–22, Berlin, Heidelberg, 2002. Springer.
46. Chris W. George, Peter Haff, Klaus Havelund, Anne Elisabeth Haxthausen, Steven Hughes, Robert Milne, Claus Bendix Nielsen, Jan Storbak Pedersen, Søren Prehn, and Kim Ritter Wagner. *The RAISE Specification Language and The RAISE Development Method*. The BCS Practitioner Series. Prentice-Hall, Hemel Hempstead, England, 1992–1995.
47. D. Gries. *The Science of Programming*. Springer-Verlag, 1981.
48. Yuri Gurevich. *Evolving Algebra 1993: Lipari Guide*, in: *Specification and Validation Methods*, pages 9–36. Oxford University Press, 1996. arXiv:1808.06255.
49. Yuri Gurevich. Sequential Abstract State Machines capture Sequential Algorithms. *ACM Transactions on Computational Logic*, 1(1):77–111, July 2000.
50. Yuri Gurevich, Benjamin Rossman, and Wolfram Schulte. Semantics Essence of AsmL. *Theoretical Computer Science*, 343(3):370–412, October 2005.
51. David Harel. Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3):231–274, 1987.
52. Klaus Havelund, Mike Lowry, and John Penix. Formal analysis of a space-craft controller using spin. *IEEE Transactions on Software Engineering*, 27(8):749–765, Aug 2001.
53. Klaus Havelund, Giles Reger, Daniel Thoma, and Eugen Zălinescu. Monitoring events that carry data. In Ezio Bartocci and Yliès Falcone, editors, *Lectures on Runtime Verification - Introductory and Advanced Topics*, volume 10457 of *LNCS*, pages 61–102. Springer, 2018.
54. Klaus Havelund, Arne Skou, Kim Larsen, and Kristian Lund. Formal modeling and analysis of an audio/video protocol: an industrial case study using uppaal. In *Proceedings Real-Time Systems Symposium*, pages 2–13, Dec 1997.
55. Klaus Havelund and Willem Visser. Program model checking as a new trend. *International Journal on Software Tools for Technology Transfer*, 4(1):8–20, Oct 2002.
56. Anne Elisabeth Haxthausen and Xia Yong. *A RAISE Specification Framework and Justification assistant for the Duration Calculus*, chapter Saarbrücken. Dept of Linguistics, Gothenburg University, Sweden, 1998.
57. Anne Elisabeth Haxthausen and Xia Yong. Linking DC together with TRSL. In *IFM'2000 2nd Int. Conf. on Integrated Formal Methods*, volume 1945 of *LNCS*, pages 25–44. Springer, 2000.
58. Ian Hayes and Bill Flinn. *Specification Case Studies*. Prentice Hall, 2nd edition, 1992.
59. Jifeng He. From CSP to Hybrid Systems. In *A Classical Mind*. Prentice Hall, 1994.
60. E.C.R. Hehner. *The Logic of Programming*. Prentice-Hall, 1984.
61. C. A. R. Hoare. The Verifying Compiler: A Grand Challenge for Computing Research. *Journal of the ACM*, 50:63–69, January 2003.
62. Charles Anthony Richard Hoare. The Axiomatic Basis of Computer Programming. *Communications of the ACM*, 12(10):567–583, Oct. 1969.
63. Charles Anthony Richard Hoare. Proof of Correctness of Data Representations. *Acta Informatica*, 1:271–281, 1972.
64. Charles Anthony Richard Hoare. *Communicating Sequential Processes*. C.A.R. Hoare Series in Computer Science. Prentice-Hall International, 1985. Published electronically: <http://www.usingcsp.com/cspbook.pdf> (2004).
65. Gerard J. Holzmann. *The SPIN Model Checker, Primer and Reference Manual*. Addison-Wesley, Reading, Massachusetts, 2003.

66. Gérard P. Huet and Hugo Herbelin. 30 years of research and development around Coq. In Suresh Jagannathan and Peter Sewell, editors, *The 41st Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL'14, San Diego, CA, USA*, pages 249–250. ACM, 2014.
67. ITU-T. CCITT Recommendation Z.120: Message Sequence Chart (MSC), 1992, 1996, 1999.
68. Daniel Jackson. Alloy: A Language and Tool for Exploring Software Designs. *Communications of the ACM*, 62(9):66–76, September 2019.
69. Daniel Jackson. *Software Abstractions: Logic, Language, and Analysis*. The MIT Press, Cambridge, Mass., USA, April 2006. ISBN 0-262-10114-9.
70. Bart Jacobs, Jan Smans, Pieter Philippaerts, Frédéric Vogels, Willem Penninckx, and Frank Piessens. VeriFast: A powerful, sound, predictable, fast verifier for C and Java. In Mihaela Gheorghiu Bobaru, Klaus Havelund, Gerard J. Holzmann, and Rajeev Joshi, editors, *NASA Formal Methods - Third International Symposium, NFM, Pasadena, CA, USA, Proceedings*, volume 6617 of *Lecture Notes in Computer Science*, pages 41–55. Springer, 2011. Tool website: <http://people.cs.kuleuven.be/~bart.jacobs/verifast>.
71. Ivar Jacobson, Grady Booch, and James Rumbaugh. *The Unified Software Development Process*. Object Technology Series. Addison-Wesley, Addison Wesley Longman, Inc., One Jacob Way, Reading, Massachusetts 01867, USA, 1999.
72. C. B. Jones, editor. *Programming Languages and Their Definition: Selected Papers of H. Bekič*. Springer, Heidelberg.
73. C. B. Jones. *Software Development: A Rigorous Approach*. Prentice-Hall, 1980.
74. Neil D. Jones, Carsten Gomard, and Peter Sestoft. *Partial Evaluation and Automatic Program Generation*. C.A.R.Hoare Series in Computer Science. Prentice Hall International, 1993.
75. K. Araki and others, editor. *IFM 1999–2013: Integrated Formal Methods*, LNCS Vols.: 1945, 2335, 2999, 3771, 4591, 5423, 6496, 7321, 7940, etc. Springer, 1999–2019.
76. Matt Kaufmann, Panagiotis Manolios, and J Strother Moore. *Computer-Aided Reasoning: An Approach*. Kluwer Academic Publishers, June 2000.
77. Donald Ervin Knuth. *The Art of Computer Programming, Vols.1-2-3: Fundamental Algorithms, Seminumerical Algorithms, Searching & Sorting*. Addison-Wesley, Reading, Mass., USA, 1968-1969. For vol.4 see: <http://freecomputerbooks.com/-The-Ar-of-Computer-Programming-Vol-4.html>.
78. C. Lakos et al., editors. *SEFM: International IEEE Conferences on Software Engineering and Formal Methods*, SEFM 2002–2013. IEEE Computer Society Press, 2002–2013.
79. Leslie Lamport. *Specifying Systems*. Addison-Wesley, Boston, Mass., USA, 2002.
80. Peter J Landin. The mechanical evaluation of expressions. *The Computer Journal*, 6(4):308–320, 1964.
81. K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In Edmund M. Clarke and Andrei Voronkov, editors, *LPAR-16*, volume 6355 of *LNCS*, pages 348–370. Springer, April 2010.
82. Zohar Manna and Amir Pnueli. *The Temporal Logic of Reactive Systems: (vol.1: Specifications, vol.2: Safety)*. Addison Wesley, 1991 and 1995.
83. John McCarthy. Recursive Functions of Symbolic Expressions and Their Computation by Machines, Part I. *Communications of the ACM*, 3(4):184–195, 1960.
84. John McCarthy. Towards a Mathematical Science of Computation. In C.M. Poplewell, editor, *IFIP World Congress Proceedings*, pages 21–28, 1962.
85. John McCarthy and James Painter. Correctness of a Compiler for Arithmetic Expressions. In *[106]*, pages 33–41, 1966. Dept. of Computer Science, Stanford University, California, USA.

86. John W. McCormick and Peter C. Chapin. *Building High Integrity Applications with SPARK*. Cambridge University Press, 2015.
87. José Meseguer. Conditional rewriting logic as a unified model of concurrency. *Theoretical Computer Science*, 96(1):73–155, 1992.
88. José Meseguer. Twenty years of rewriting logic. *J. Logic and Algebraic Programming*, 81:721–781, 2012.
89. R. Milner. *Calculus of Communication Systems*, volume 94 of *Lecture Notes in Computer Science*. Springer, 1980.
90. Robin Milner, Mads Tofte, Robert Harper, and David McQueen. *The Definition of Standard ML, Revised*. The MIT Press, Cambridge, Mass., USA and London, England, 1997.
91. Leonardo De Moura and Nikolaj Bjørner. Z3: An efficient smt solver. In *Proceedings of the Theory and Practice of Software, 14th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, TACAS'08/ETAPS'08*, pages 337–340, Berlin, Heidelberg, 2008. Springer-Verlag.
92. Silvia M. Müller and Wolfgang J. Paul. *Computer Architecture, Complexity and Correctness*. Springer, Berlin, Heidelberg, 2000.
93. Chris Newcombe, Tim Rath, Fan Zhang, Bogdan Munteanu, Marc Brooker, and Michael Deardeuff. How Amazon Web Services uses formal methods. *Communications of the ACM*, 58(4):66–73, April 2015.
94. Tobias Nipkow, Lawrence C. Paulson, and Markus Wenzel. *Isabelle/HOL, A Proof Assistant for Higher-Order Logic*, volume 2283 of *Lecture Notes in Computer Science*. Springer-Verlag, 2002.
95. Ulf Norell. Dependently typed programming in agda. In Pieter W. M. Koopman, Rinus Plasmeijer, and S. Doaitse Swierstra, editors, *Advanced Functional Programming, 6th International School, AFP 2008, Heijen, The Netherlands, May 2008, Revised Lectures*, volume 5832 of *Lecture Notes in Computer Science*, pages 230–266. Springer, 2008.
96. Ole N. Oest. Vdm from research to practice (invited paper). In *IFIP Congress*, pages 527–534, 1986.
97. S. Owre, J.M. Rushby, and N. Shankar. PVS: A prototype verification system. In D. Kapur, editor, *11th International Conference on Automated Deduction (CADE), Saratoga – NY, Lecture Notes in Artificial Intelligence, Volume 607*, pages 748–752. Springer Verlag, June 1992.
98. Sam Owre, John Rushby, Natarajan Shankar, and Friedrich von Henke. Formal verification for fault-tolerant architectures: Prolegomena to the design of PVS. *IEEE Trans. Software Eng.*, 21(2):107–125, February 1995. PVS home page: <http://pvs.csl.sri.com>.
99. Wolfgang J. Paul. Towards a Worldwide Verification Technology. In Bertrand Meyer and Jim Woodcock, editors, *Verified Software: Theories, Tools, Experiments*, volume 4171 of *Lecture Notes in Computer Science*, pages 19–25, Berlin, Heidelberg, 2005. Springer.
100. Wolfgang J. Paul, Christoph Baumann, Petro Lutsyk, and Sabine Schmaltz. *System Architecture - An Ordinary Engineering Discipline*. Springer, Berlin, Heidelberg, 2016.
101. Wolfgang Reisig. Abstract State Machines for the Classroom. In Dines Bjørner and Martin C. Henson, editors, *Logics of Specification Languages, EATCS*, pages 15–46. Springer, 2008.
102. Wolfgang Reisig. *Understanding Petri Nets Modeling Techniques, Analysis Methods, Case Studies*. Springer, 2013. 230+XXVII pages, 145 illus.
103. John C Reynolds. Definitional interpreters for higher-order programming languages. In *Proceedings of the ACM annual conference-Volume 2*, pages 717–740. ACM, 1972.

104. John C Reynolds. *The Craft of Programming*. Prentice Hall PTR, 1981.
105. Donald Sannella and Andrzej Tarlecki. *Foundations of Algebraic Semantics and Formal Software Development*. Monographs in Theoretical Computer Science. Springer, Heidelberg, 2012.
106. Jack T. Schwartz. *Mathematical Aspects of Computer Science, Proc. of Symp. in Appl. Math.* American Mathematical Society, Rhode Island, USA, 1967.
107. Dana S. Scott and Christopher Strachey. Towards a mathematical semantics for computer languages. In *Computers and Automata*, volume 21 of *Microwave Research Inst. Symposia*, pages 19–46, 1971.
108. Peter Sestoft. *Java Precisely*. The MIT Press, 25 July 2002.
109. J. Michael Spivey. *The Z Notation — A Reference Manual*. Series in Computer Science. Prentice Hall, 2nd edition, 1992.
110. Bernhard Steffen, Frederik Gossen, Stefan Naujokat, and Tiziana Margaria. Language-Driven Engineering: From General-Purpose to Purpose-Specific Languages. In Bernhard Steffen and Gerhard Woeginger, editors, *Computing and Software Science: State of the Art and Perspectives*, volume 10000 of *LNCS*. Springer, 2018.
111. Simon Thompson. *Haskell: The Craft of Functional Programming*. Addison Wesley, 2nd edition, 29 March 1999. 512 pages, ISBN 0201342758.
112. Mark Utting and Bruno Legeard. *Practical model-based testing: a tools approach*. Elsevier, 2010.
113. Peter J. Whysall and John A. McDermid. An approach to object-oriented specification using Z. In John E. Nicholls, editor, *Z User Workshop, Oxford 1990*, Workshops in Computing, pages 193–215. Springer, 1991.
114. James Charles Paul Woodcock and James Davies. *Using Z: Specification, Refinement, and Proof*. Prentice Hall., 1996.
115. WanLing Xie, ShuangQing Xiang, and HuiBiao Zhu. A UTP approach for rTiMo. *Formal Aspects of Computing*, 30:713–738, 2018.
116. Naijun Zhan, Shuling Wang, and Hengjun Zhao. Formal modelling, analysis and verification of hybrid systems. In *ICTAC Training School on Software Engineering*, pages 207–281, 2013.
117. Chao Chen Zhou and Michael R. Hansen. *Duration Calculus: A Formal Approach to Real-time Systems*. Monographs in Theoretical Computer Science. An EATCS Series. Springer-Verlag, 2004.